FH Salzburg
MultiMediaTechnology

Basics Game Development

Game Objects, Collisions, Physics, - Lab

Technology
Health
Media

10 Minute Quiz

Assignment 2 Presentation

Today's Topics

- GameObject Class
- Simple Collision Detection

GameObject Class – Approach with Abstract Class

- Create an abstract base class - GameObject
- Abstract classes can't be instantiated on its own
- Abstract classes may contain abstract methods and accessors
- An abstract method is implicitly a virtual method
- Derived non-abstract classes must implement members / methods from the abstract class

```
internal abstract class GameObject : Transformable
{
    public GameObject() { }

    public abstract void Update(float deltaTime);
    public abstract void Draw(RenderWindow window);
}
```

GameObject – Player Class example

- This is a sample of player class which is derived from the GameObject class
- We can implement Player specific logic in the override methods

```
internal class Player : GameObject
{
    private readonly Sprite animatedSprite;

    public Player()
    {
        // Player Init Logic

        // Load Texture
        // Set Sprite
        // Set Collider
        // ...
    }

    public override void Update(float deltaTime)
    {
        // Player Update Logic

        // Handle Input
        // Move Player
        // ...
    }

    public override void Draw(RenderWindow window)
    {
        // Player Drawing Logic

        // Draw Animation Frame
        // ...

        window.Draw(animatedSprite);
    }
}
```

GameObject – example usage

- We can use a List of GameObjects (derived objects; i.e player, enemy, obstacles, etc.)

```
private List<GameObject> gameObjects = new();
```

- Simply call the virtual methods of the GameObject class for each instantiated derived class

```
private void Update(float deltaTime)
{
        foreach (var gameObject in gameObjects)
          gameObject.Update(deltaTime);
}
```

```
private void Draw()
{
    window.Clear(BG_COLOR);

    foreach (var gameObject in gameObjects)
        gameObject.Draw(window);

    window.Display();
}
```

Collision – SFML Rectangle

- Detect a Rectangle intersection with SFML

```
internal class Player : GameObject
{
    13 references
    public IntRect CollisionRect { get; private set; }
```

```
if(player.CollisionRect.Intersects(otherColliderRect))
{
    // Do Collision Logic here
}
```

Simple player collision detection

- Add Objects to a List (e.g., Obstacles, Projectiles, Collectables, etc.)
- In the Game.Update() loop over all relevant Objects to check for intersection

```
// Loop over all Obstacles to check for intersection
foreach (Obstacle obstacle in obstacles)
{
    if (player.CollisionRect.Intersects(obstacle.CollisionRect))
    {
        // Do Logic if Player Intersects with Obstacle
    }
}
```

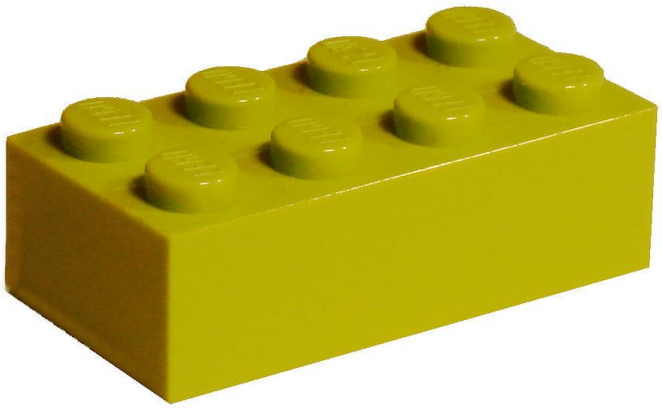
Collision – AABB & Circle

- Code aus der Vorlesung

```
bool Circle_AABB_Collision(CircleShape circle, RectangleShape rect)
{
    var getCenter = (Shape shape)=>{return new Vector2f(){
        X= shape.GetGlobalBounds().Left + shape.GetGlobalBounds().Width / 2f,
        Y= shape.GetGlobalBounds().Top + shape.GetGlobalBounds().Height / 2f};};

    Vector2f circleDistance = new();
    var centerCircle = getCenter(circle);
    var centerRect = getCenter(rect);

    circleDistance.X = MathF.Abs(centerCircle.X - centerRect.X);
    circleDistance.Y = MathF.Abs(centerCircle.Y - centerRect.Y);

    if (circleDistance.X> (rect.Size.X / 2 + circle.Radius)) { return false; }
    if (circleDistance.X > (rect.Size.Y / 2 + circle.Radius)) { return false; }

    if (circleDistance.X <= (rect.Size.X / 2)) { return true; }
    if (circleDistance.Y <= (rect.Size.Y / 2)) { return true; }

    var cornerDistance_sq = (circleDistance.X - rect.Size.X / 2) * (circleDistance.X - rect.Size.X / 2) +
                            (circleDistance.Y - rect.Size.Y / 2) * (circleDistance.Y - rect.Size.Y / 2);

    return (cornerDistance_sq <= (circle.Radius *circle.Radius));
}
```

Note

This is an intermediate step on the way to a component-based architecture. In the final architecture, the Game-Object would only act as a container for components like PhysicsComponent, SpriteComponent, ColliderComponent, SoundComponent, PlayerMoveComponent, etc.



Assignment 3

FH Salzburg
MultiMediaTechnology

Technology
Health
Media

© 2025 Fachhochschule Salzburg GmbH
www.fh-salzburg.ac.at